From Parallel Lines to Concentric Rings

Buffon's Needle on Curved Boundaries

Vijay · Independent project · October 2026
Code repository: [insert repository link]

Abstract

Buffon's needle estimates π from the rate at which a needle dropped on parallel lines crosses one. I replace the parallel lines with concentric circles of spacing D and drop a needle of length $L = D$ with its centre uniform by area on an annulus. A Monte Carlo simulation (Java, with a NumPy cross-check) shows that curvature lowers the crossing probability slightly below the straight-line value $2/\pi \approx 0.63662$. The gap is about 0.004 for needles centred in the first ring and falls below the statistical noise (about 0.0003) by roughly the fifth ring. For an annulus out to $R_{max} = 5D$ the crossing ratio is 0.63533 ± 0.00011, below $2/\pi$ by about 0.0013. For $R_{max} \geq 100D$ the ratio is statistically indistinguishable from $2/\pi$. I also show that testing only the needle's endpoints undercounts crossings near the centre, and I give an exact test.

Plain-language overview

In 1733 Buffon showed that dropping a stick on a floor of parallel lines lets you estimate π: the fraction of drops that cross a line is $2L/(\pi D)$.

This project asks: what if the lines are concentric circles, like a dartboard? Near the centre the circles are small and sharply curved, so the stick behaves differently. Far from the centre the circles are so large that, up close, they look like straight lines.

I simulated millions of drops. Curvature does change the answer, but only near the centre, and only slightly. Because the outer region holds almost all of the area, a big dartboard behaves almost exactly like the original floor.

1. Background

For a needle of length L dropped on parallel lines spaced $D \geq L$ apart, the crossing probability is

$$P = 2L / (\pi D)$$

With $L = D$ this is $2/\pi \approx 0.636620$, so $\pi \approx 2N/C$ for N drops and C crossings. This paper keeps $L = D = 3.8$ cm and changes only the geometry of the lines.

Buffon-type problems for other shapes and lattices belong to the field of integral geometry (see Santaló, *Integral Geometry and Geometric Probability*). I have not found a treatment of this exact setup, but I have not done a full literature search, so I make no claim of novelty.

2. Setup

Geometry. Rings are the circles of radius kD, $k = 1, 2, \ldots$ The needle's centre is at distance r from the origin, and θ is the angle between the needle and the radial direction. The distances from the origin to the two endpoints follow from the law of cosines:

$$r_{1,2} = \sqrt{(r^2 + (L/2)^2 \pm r\, L \cos\theta)}$$

Crossing condition. Distance from the origin varies continuously along the needle. So the needle crosses a ring boundary exactly when the interval $[r_{min}, r_{max}]$ of distances along it contains a multiple of D, that is,

$$\lfloor r_{min} / D \rfloor \neq \lfloor r_{max} / D \rfloor$$

where $r_{max} = \max(r_1, r_2)$ and r_{min} is the shortest distance from the origin to the needle segment. I call this the **exact test**.

A simpler **endpoint test** compares $\lfloor r_1/D \rfloor$ with $\lfloor r_2/D \rfloor$. It misses needles lying almost tangent to a ring: both endpoints are in the same ring, but the middle dips into the next ring in. The endpoint test was in my first version of the code. Section 3 measures how much it matters.

Sampling.

- *Centre:* uniform by area on the annulus $[R_{min}, R_{max}]$ with $R_{min} = D$, using inverse-CDF sampling: $r = \sqrt{(R_{min}^2 + u(R_{max}^2 - R_{min}^2))}$, $u \sim \mathrm{U}(0,1)$.
- *Direction:* unit-disk rejection sampling. Draw (d_x, d_y) uniform in $[-1,1]^2$ until $0 < d_x^2 + d_y^2 \le 1$, then $\cos\theta = d_x / \sqrt{(d_x^2 + d_y^2)}$. This uses neither `Math.PI` nor trigonometric functions, so π does not enter the simulation.
- *Window:* the drops cover a finite annulus, not the whole plane. With area weighting and $R_{max} = 10^4 D$, only about 10^{-6} of drops fall within $10D$ of the centre.

Software. `ConcentricBuffon.java` (Java, `java.util.Random`, seeded) computes both tests. `sim.py`, `sweep.py` and `conv.py` (NumPy) produced the sweeps below. The Java and NumPy results agree within noise. The full source is in Section 6.

3. Results

The statistical error on a ratio from n drops is about $0.481/\sqrt{n}$, i.e. 0.00034 for $n = 2\times10^6$.

3.1 Crossing probability ring by ring

Needle centre drawn uniformly by area within ring k, i.e. $r \in [kD, (k+1)D)$; $n = 2\times10^6$ per row; $2/\pi = 0.636620$.

Ring k	Endpoint test	Exact test	Exact − 2/π
1	0.62700	0.63257	−0.0040
2	0.63251	0.63448	−0.0021
3	0.63470	0.63571	−0.0009
5	0.63588	0.63628	−0.0003
10	0.63618	0.63630	−0.0003
20	0.63648	0.63651	−0.0001
50	0.63691	0.63691	+0.0003
100	0.63653	0.63653	−0.0001
1000	0.63675	0.63675	+0.0001

Curvature lowers the crossing probability, most strongly in the innermost rings. From about $k = 5$ onward the exact-test deviation is within about one standard error. The exact test is higher than the endpoint test in every inner row, and the two tests coincide for $k \ge 50$.

3.2 Whole-annulus ratio versus R_{max}

Centres area-uniform on $[D, R_{max}]$; 10 seeds of 2×10^6 drops each; mean ± standard deviation across seeds.

R_{max}/D	Endpoint test	Exact test	Exact − 2/π	Std. error of mean
5	0.63369 ± 0.00036	0.63533 ± 0.00036	−0.00129	0.00011
10	0.63563 ± 0.00022	0.63621 ± 0.00022	−0.00041	0.00007
100	0.63663 ± 0.00039	0.63664 ± 0.00039	+0.00002	0.00012
10,000	0.63679 ± 0.00033	0.63679 ± 0.00033	+0.00017	0.00010

Small annuli show a clear, significant shortfall relative to 2/π. At $R_{max} = 100D$ and above, the exact-test mean is within about two standard errors of 2/π. These data show the deviation shrinking as R_{max} grows. They do not establish a convergence rate.

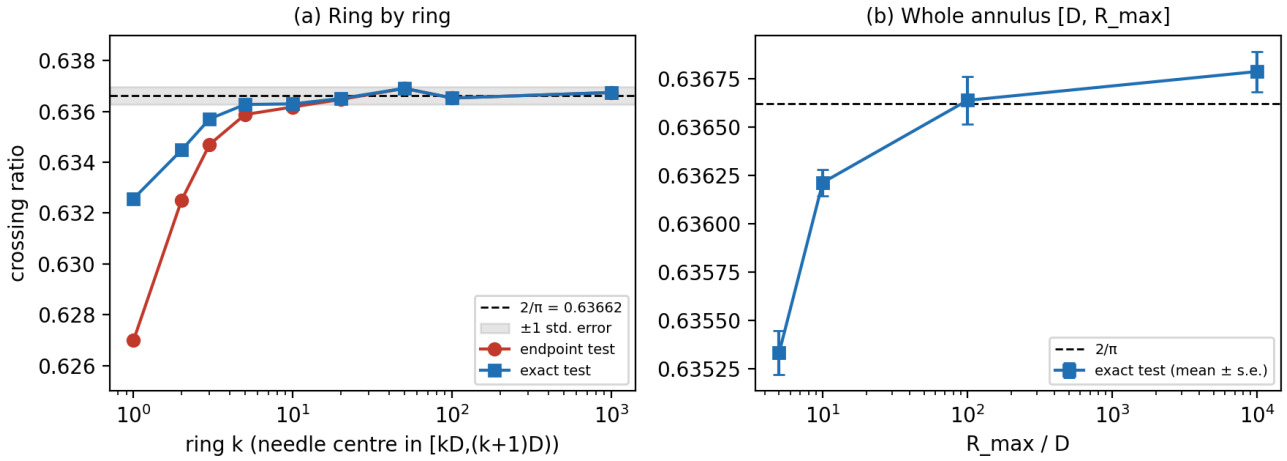


Figure 1. (a) Crossing ratio by ring for the two tests; the shaded band is ±1 standard error around 2/π. (b) Whole-annulus exact-test ratio versus R_{max} (mean ± standard error of 10 seeds).

3.3 Estimating π and how many drops it takes

Using $R_{max} = 10^4 D$ and the exact test (identical to the endpoint test here). Error is $2N/C - \pi$, over 20 seeds (5 seeds for $N = 10^7$). Theory predicts a standard error of about $2.37/\sqrt{N}$.

N	Mean |error|	RMS error	Theory $2.37/\sqrt{N}$
10^3	0.0726	0.097	0.075
10^4	0.0183	0.023	0.024
10^5	0.0053	0.0063	0.0075
10^6	0.0015	0.0019	0.0024
10^7	—	0.0009	0.00075

Five runs at $N = 10^7$ gave π estimates of 3.14093, 3.14256, 3.14263, 3.14065 and 3.14100. A single run can land very close to π by chance, but the typical error at this size is about ±0.0008, so only about three digits are reliable.

4. Discussion

- **Curvature has a real, measurable effect.** The crossing probability is below 2/π when needles land near the centre (Sections 3.1 and 3.2).
- **The big-annulus result mostly reproduces classical Buffon.** With area-uniform drops out to $10^4 D$, nearly every drop lands where the rings are almost straight. The π estimate in Section 3.3 therefore confirms the implementation and the classical result, not a new effect. The informative experiments are Sections 3.1 and 3.2, where the curvature is visible.
- **The endpoint test is biased low near the centre.** For $R_{max} = 5D$ it undercounts by about 0.0016 compared with the exact test. A study that only compares endpoints would mistake part of this for a curvature effect, so the exact test should be used.
- **No rate or proof.** I give no analytic form for the deviation and no convergence rate in r/L. A closed-form crossing probability at fixed r is left as future work and would explain the shape in Section 3.1.

5. Limitations

- Finite sampling window; the whole-plane limit is approached, not reached.

- Pseudorandom numbers (`java.util.Random`, NumPy's default generator), not physical randomness.
- A single value $L = D$; dependence on L/D was not tested.
- Deviations below about 0.0003 are at the noise level of these experiments.
- The needle is modelled as an ideal line segment.

6. Source code

All code used for the results in this paper. Lines longer than 100 characters are wrapped with extra indentation for printing. Run the Java program with `java ConcentricBuffon.java <drops> <Rmax/D> <seed>`, for example `java ConcentricBuffon.java 5000000 5 1`.

6.1 ConcentricBuffon.java (main simulation, both crossing tests)

```
import java.util.Random;

/** Buffon's needle on concentric rings. Usage: java ConcentricBuffon.java <drops> <RmaxOverD>
    <seed> */
public class ConcentricBuffon {
    public static void main(String[] args) {
        final double L = 3.8, D = 3.8, h = L / 2.0;
        long n = Long.parseLong(args[0]);
        double rMax = Double.parseDouble(args[1]) * D;
        long seed = Long.parseLong(args[2]);
        final double rMin = D;

        Random rnd = new Random(seed);
        long endpointHits = 0, exactHits = 0;

        for (long i = 0; i < n; i++) {
            // centre radius: uniform by area on the annulus [rMin, rMax]
            double r = Math.sqrt(rMin * rMin + rnd.nextDouble() * (rMax * rMax - rMin * rMin));

            // direction relative to the radial line: unit-disk rejection sampling (no pi, no trig)
            double dx, dy, s;
            do {
                dx = 2.0 * rnd.nextDouble() - 1.0;
                dy = 2.0 * rnd.nextDouble() - 1.0;
                s = dx * dx + dy * dy;
            } while (s > 1.0 || s == 0.0);
            double len = Math.sqrt(s);
            double c = dx / len;

            // endpoint radii (law of cosines)
            double r1 = Math.sqrt(r * r + h * h + 2 * r * h * c);
            double r2 = Math.sqrt(r * r + h * h - 2 * r * h * c);
            if (Math.floor(r1 / D) != Math.floor(r2 / D)) endpointHits++;

            // exact test: radius varies continuously along the stick, so it crosses a ring
            // boundary iff [min radius, max radius] contains a multiple of D
            double t = Math.max(-h, Math.min(h, -r * c));
            double rLow = Math.sqrt(Math.max(r * r + 2 * r * t * c + t * t, 0.0));
            double rHigh = Math.max(r1, r2);
            if (Math.floor(rLow / D) != Math.floor(rHigh / D)) exactHits++;
        }
        System.out.printf("drops=%d Rmax=%.0fD seed=%d%n", n, rMax / D, seed);
        System.out.printf("endpoint test: crossings=%d ratio=%.6f pi_est=%.5f%n", endpointHits,
            (double) endpointHits / n, 2.0 * n / endpointHits);
        System.out.printf("exact test:    crossings=%d ratio=%.6f pi_est=%.5f%n", exactHits,
            (double) exactHits / n, 2.0 * n / exactHits);
    }
}
```

6.2 sim.py (NumPy implementation and ring-by-ring experiment)

```
import numpy as np, sys
L=D=3.8; h=L/2; TWO_OVER_PI=2/np.pi

def directions(rng,n):
    # unit-disk rejection sampling (no trig, no pi)
    cs=np.empty(n); ss=np.empty(n); got=0
    while got<n:
        m=int((n-got)*1.4)+16
        dx=2*rng.random(m)-1; dy=2*rng.random(m)-1; s=dx*dx+dy*dy
        k=(s<=1)&(s>0); dx,dy,s=dx[k],dy[k],s[k]
        l=np.sqrt(s); take=min(len(dx),n-got)
        cs[got:got+take]=dx[:take]/l[:take]; ss[got:got+take]=dy[:take]/l[:take]; got+=take
    return cs,ss

def crossings(rng,n,rlo,rhi,chunk=2_000_000):
    """returns (endpoint_test_count, exact_test_count). center r sampled uniform by area on
        [rlo,rhi]"""
    ce=cx=0; done=0
    while done<n:
        m=min(chunk,n-done)
        u=rng.random(m); r=np.sqrt(rlo**2+u*(rhi**2-rlo**2))
        c,s=directions(rng,m)
        r1=np.sqrt(r*r+h*h+2*r*h*c); r2=np.sqrt(r*r+h*h-2*r*h*c)
        ce+=np.count_nonzero(np.floor(r1/D)!=np.floor(r2/D))
        t=np.clip(-r*c,-h,h); rmin=np.sqrt(np.maximum(r*r+2*r*t*c+t*t,0))
        rmax=np.maximum(r1,r2)
        cx+=np.count_nonzero(np.floor(rmin/D)!=np.floor(rmax/D))
        done+=m
    return ce,cx

if __name__=="__main__":
    rng=np.random.default_rng(12345)
    n=2_000_000
    print("A) ring-by-ring: center drawn in ring k = [kD,(k+1)D), n=%d each; 2/pi=%.6f,
        SE~%.5f"%(n,TWO_OVER_PI,0.4811/np.sqrt(n)))
    for k in [1,2,3,5,10,20,50,100,1000]:
        ce,cx=crossings(rng,n,k*D,(k+1)*D)
        print(f"k={k:5d} endpoint={ce/n:.6f} exact={cx/n:.6f}")
```

6.3 sweep.py (R_max sweep and 10^7-drop runs)

```
import numpy as np, time
from sim import *
t0=time.time()
n=2_000_000; seeds=10
print("B) annulus [D, Rmax], n=%d per run, %d seeds"%(n,seeds))
for R in [5,10,100,10000]:
    pe=[];px=[]
    for s in range(seeds):
        rng=np.random.default_rng(1000+s)
        ce,cx=crossings(rng,n,D,R*D); pe.append(ce/n); px.append(cx/n)
    pe=np.array(pe);px=np.array(px)
    print(f"Rmax={R:6d}D endpoint mean={pe.mean():.6f} sd={pe.std(ddof=1):.6f} | exact
        mean={px.mean():.6f} sd={px.std(ddof=1):.6f} | exact-2/pi={px.mean()-TWO_OVER_PI:+.6f} |
        pi_est(exact mean)={2/px.mean():.5f}")
print("time",time.time()-t0)
# one big run like the paper: 1e7 drops, Rmax=10^4 D, 5 seeds
N=10_000_000
print("C) N=1e7, Rmax=1e4 D, 5 seeds (exact test)")
for s in range(5):
    rng=np.random.default_rng(2000+s)
    ce,cx=crossings(rng,N,D,10000*D)
    print(f"seed {2000+s}: endpoint ratio={ce/N:.6f} pi={2*N/ce:.5f} | exact ratio={cx/N:.6f}
        pi={2*N/cx:.5f}")
print("time",time.time()-t0)
```

6.4 conv.py (convergence in N)

```
import numpy as np
from sim import *
print("D) convergence with N (Rmax=1e4 D, exact test), 20 seeds each: mean |pi_est - pi|, and theory
    SE of pi")
for N in [1000,10000,100000,1000000]:
    errs=[]
    for s in range(20):
        rng=np.random.default_rng(3000+s); _,cx=crossings(rng,N,D,10000*D);
            errs.append(2*N/cx-np.pi)
    e=np.array(errs); print(N, f"mean|err|={np.abs(e).mean():.5f} rms={np.sqrt((e**2).mean()):.5f}")
```

7. Program output

Raw output of the programs above, from the runs that produced the tables in Section 3. The Java run with $R_{\max} = 5D$ shows the endpoint test undercounting (3,170,945 versus 3,179,119 crossings) on the same random drops.

7.1 Ring-by-ring (sim.py)

```
$ python3 sim.py
A) ring-by-ring: center drawn in ring k = [kD,(k+1)D), n=2000000 each; 2/pi=0.636620, SE~0.00034
k=    1 endpoint=0.626997 exact=0.632572
k=    2 endpoint=0.632507 exact=0.634481
k=    3 endpoint=0.634703 exact=0.635709
k=    5 endpoint=0.635879 exact=0.636275
k=   10 endpoint=0.636179 exact=0.636300
k=   20 endpoint=0.636477 exact=0.636512
k=   50 endpoint=0.636907 exact=0.636912
k=  100 endpoint=0.636530 exact=0.636530
k= 1000 endpoint=0.636748 exact=0.636748
```

7.2 R_max sweep and 10^7-drop runs (sweep.py)

```
$ python3 sweep.py
B) annulus [D, Rmax], n=2000000 per run, 10 seeds
Rmax=     5D endpoint mean=0.633691 sd=0.000356 | exact mean=0.635333 sd=0.000359 |
    exact-2/pi=-0.001287 | pi_est(exact mean)=3.14796
Rmax=    10D endpoint mean=0.635627 sd=0.000224 | exact mean=0.636213 sd=0.000215 |
    exact-2/pi=-0.000407 | pi_est(exact mean)=3.14360
Rmax=   100D endpoint mean=0.636627 sd=0.000387 | exact mean=0.636638 sd=0.000386 |
    exact-2/pi=+0.000019 | pi_est(exact mean)=3.14150
Rmax= 10000D endpoint mean=0.636787 sd=0.000328 | exact mean=0.636787 sd=0.000328 |
    exact-2/pi=+0.000167 | pi_est(exact mean)=3.14077
C) N=1e7, Rmax=1e4 D, 5 seeds (exact test)
seed 2000: endpoint ratio=0.636754 pi=3.14093 | exact ratio=0.636754 pi=3.14093
seed 2001: endpoint ratio=0.636424 pi=3.14256 | exact ratio=0.636424 pi=3.14256
seed 2002: endpoint ratio=0.636409 pi=3.14263 | exact ratio=0.636409 pi=3.14263
seed 2003: endpoint ratio=0.636810 pi=3.14065 | exact ratio=0.636810 pi=3.14065
seed 2004: endpoint ratio=0.636740 pi=3.14100 | exact ratio=0.636740 pi=3.14100
```

7.3 Convergence in N (conv.py)

```
$ python3 conv.py
D) convergence with N (Rmax=1e4 D, exact test), 20 seeds each
1000 mean|err|=0.07255 rms=0.09693
10000 mean|err|=0.01829 rms=0.02266
100000 mean|err|=0.00534 rms=0.00634
1000000 mean|err|=0.00152 rms=0.00190
```

7.4 Java program

```
$ java ConcentricBuffon.java 10000000 10000 1
drops=10000000 Rmax=10000D seed=1
endpoint test: crossings=6367234 ratio=0.636723 pi_est=3.14108
exact test:    crossings=6367234 ratio=0.636723 pi_est=3.14108

$ java ConcentricBuffon.java 5000000 5 1
drops=5000000 Rmax=5D seed=1
endpoint test: crossings=3170945 ratio=0.634189 pi_est=3.15363
exact test:    crossings=3179119 ratio=0.635824 pi_est=3.14553
```

References

1. Buffon, G.-L. L. (1777). *Essai d'arithmétique morale.* In *Supplément à l'Histoire Naturelle*, vol. 4. (Problem first presented to the Académie des Sciences in 1733.)

2. Laplace, P.-S. (1812). *Théorie analytique des probabilités.* Paris: Courcier.

3. Santaló, L. A. (1976). *Integral Geometry and Geometric Probability.* Addison-Wesley.